INDEX

Sr No.	Title	Date	Sign
	Module-1		
1	**Creation of Test Suite Using Selenium IDE** **Tool: Selenium IDE (Chrome/Firefox Extension)** Install Selenium IDE and create a test suite containing a minimum of four test cases to validate different web page formats (HTML page, XML response, JSON API output, and a dynamic form page). Execute the suite and export the automation script in Web Driver format.	**22/6/26**	
2	**Cross-Website Functional Testing Using Selenium IDE** **Tool: Selenium IDE** Develop and execute a test suite for two different websites. Perform functional operations such as clicking links, filling forms, selecting dropdown options, and Verifying displayed content. Analyze test results and generate execution logs.	**25/6/26**	
3	**Manual Test Case Design and Execution** **Tool: MS Excel / Google Sheets** Prepare a Test Plan, Test Scenarios, and Test Cases for a given web application. Execute test cases manually and prepare a Test Execution Report and Defect Report as per STLC process.	**2/7/27**	
4	**Selenium Server (Grid) Configuration and Remote Execution** **Tool: Selenium Server (Grid)** Install and configure Selenium Server in Hub-Node architecture. Execute a test script on multiple browsers using Remote WebDriver and compare cross-browser results.	**9/7/26**	
6	**Data-Driven Testing Using Excel Integration** **Tool: Selenium WebDriver + Apache POI** Write a program to update 10 student records in an Excel file and validate changes on the web application. Perform data reading, writing, and verification using Apache POI library.	**14/7/26**	

7	**Data Extraction and Score Analysis from Web Table** **Tool**: Selenium WebDriver Automate extraction of student marks from a dynamic web table and determine the number of students scoring above 60 in at least one subject. Compute total count and percentage.	**16/7/26**	
8	**Object Identification and Counting on Web Page** **Tool: Selenium WebDriver** Write a program to identify and count the total number of objects present on a web page including links, images, buttons, and input elements. Display categorized counts.	**20/7/26**	
9	**List and Combo Box Item Verification** **Tool: Selenium WebDriver** Automate the process of identifying a list or dropdown box on a web page and determine the total number of items. Validate default selected value and perform selection testing.	**24/7/26**	
	Module-2		
1	**Checkbox Identification and Validation** **Tool: Selenium WebDriver** Write a program to count the total number of checkboxes on a web page, including checked and unchecked counts. Perform dynamic selection and validation.	**27/7/26**	
2	**Dynamic Web Table Handling and Sorting Validation** **Tool: Selenium WebDriver** Automate extraction of data from a dynamic web table. Validate sorting functionality and verify correctness of filtered results.	**3/8/26**	
3	**File Upload and Download Automation** **Tool: Selenium WebDriver + Robot Class** Develop a script to automate file upload and download functionality. Validate successful upload and verify file existence after download.	**7/8/26**	
4	**Screenshot Capture and Logging Mechanism** **Tool**: **Selenium WebDriver + Log4j** Implement screenshot capture on test failure and integrate logging mechanism using Log4j. Generate execution logs and analyze error messages.	**13/8/26**	
5	**Page Object Model (POM) Framework Implementation** **Tool: Selenium WebDriver + TestNG** Design and implement automation framework using Page Object Model (POM).Create reusable page classes and execute modular test cases.	**20/8/26**	
6.	**TestNG Framework Implementation and Reporting** **Tool: Selenium WebDriver + TestNG** Create and execute automated test cases using TestNG	**27/8/26**	

	annotations. Group test cases, define priorities, and generate HTML test reports.		
7	**Continuous Integration using Jenkins** **Tool: Jenkins + Selenium WebDriver** Configure Jenkins to execute Selenium automation scripts automatically. Generate build reports and analyze test execution history.	**8/9/26**	
8	**Cross-Browser Testing Using Selenium Grid** **Tool: Selenium Grid** Execute automation scripts across multiple browsers (Chrome, Firefox, Edge) using Selenium Grid. Compare execution time and compatibility issues.	**22/9/26**	

Module -1

PRACTICAL NO-1

AIM: Install Selenium IDE and create a test suite containing a minimum of 4 test cases for different web pages formats (e.g, HTML, XML, JSON, etc.)

Steps

1. Launch Mozilla Firefox Browser.

2. Type URL https://www.seleniumhq.org/download/

OR

3. https://addons.mozilla.org/en-US/firefox/addon/selenium-ide/

4. Selenium IDE Add-ons page will get open then Click on Add to Firefox button.
5. Firefox will show one popup saying do you want to allow Mozilla Firefox to install Selenium IDE Add-on. Click on Install button.
6. Firefox will automatically install Selenium IDE software. After the installation is completed, a window appears asking to re-start the Firefox. Click on the “Restart Now” button to reflect the Selenium IDE installation. Click on Restart Now button.
7. On clicking on the Restart Now button, Firefox will restart automatically. In case you missed the restart, simply close the Firefox and launch again.
8. Once the Firefox is booted and started again, we can see Selenium IDE under the Tools menu list. IDE icon will be displayed in the Firefox toolbar.

2. 4 test cases for 4 websites

(i) http://store.demoqa.com
(ii) https://www.seleniumhq.org
(iii) http://www.google.com
(iv) http://www.yahoo.com
(v) http://demo.guru99.com/test/newtours/

Step 1

- Launch Firefox and Selenium IDE.
- Type the value for our Base URL: http://demo.guru99.com/test/newtours/.
- Toggle the Record button on (if it is not yet toggled on by default).

Step 2
In Firefox, navigate to http://demo.guru99.com/test/newtours/.

Step 3

- Right-click on any blank space within the page, like on the Mercury Tours logo on the page, to bring up the Selenium IDE context menu. Note: Do not click on any hyperlinked object.
- Select the "Show Available Commands" option.
- Then, select "assertTitle exact: Welcome: Mercury Tours." This is a command that is correct

Step 4

- In the "User Name" text box of Mercury Tours, type an invalid username, "invalidU".
- In the "Password" text box, type an invalid password, "invalidPWD".

Step 5
Click on the "Sign-In" button. Firefox should take you to this page.

Step 6 Toggle the record button off to stop recording.

Step 7

Now that we are done with our test script, we shall save it in a test case.

Step 8

- Choose your desired location, and then name the Test Case as “Invalid_login”.
- Click the “Save” button.

Step 9

Notice that the file was saved as side extension.

PRACTICAL NO -2

AIM: Conduct a test suite for two different websites using selenium IDE. Perform various actions like clicking links, filling forms, and verifying contents.

TC'S 1 : Manual Steps

1. Open (Example : Type www.google.com)

2. Type “energy efficient” in the Google Search Input Box

3. Click outside on an empty spot
4. Click Search Button
5. Verify the Text Present as “energy efficient”
6. Assert the Title as “energy efficient-Google Search”

7. Save the test case with .HTML Extension.

TC'S 2 : Manual Steps

1. Open Firefox Web Browser.
2. In the address bar, Type http://www.yahoo.com

3. In the search input button, Type “energy efficient”.

4. Click on “Web Search” submit button

5. Wait for Search Results to come on “http://search.yahoo.com.”

6. Verify “energy efficient” text is present anywhere in the search results:

(Select and highlight anywhere in the search results page, “energy efficient” text is present.)

7. Verify the browsers title has the value “energy efficient - Yahoo! Search Results”.

8. Save the test case with .HTML Extension.

Steps for creating test suite

1. Create more Tc's save each Test Case with
2. Open Firefox
3. Open Tools → Selenium IDE
4. File → Open → new Test Suite
5. File → Open → Add Test cases
6. Add more test cases
7. Save Suite with <Html> extensions.
8. Run the test su

PRACTICAL NO-3

AIM: To prepare a Test Plan, Test Scenarios, and Test Cases for a given web application, execute the test cases manually, and prepare a Test Execution Report and Defect Report according to the Software Testing Life Cycle (STLC) process.

Tool Required

- MS Excel / Google Sheets
- Web Browser
- Web Application under Test

Step 1:Login page Code

```
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>Login Application</title>

    <style>
        body {
            font-family: Arial, sans-serif;
            background-color: #f2f2f2;
            display: flex;
            justify-content: center;
            align-items: center;
            height: 100vh;
        }
        input {
            width: 100%;
            padding: 10px;
            margin-top: 5px;
            box-sizing: border-box;
            border: 1px solid #ccc;
            border-radius: 5px;
        }

        button {
            width: 100%;
            padding: 10px;
            margin-top: 20px;
            background-color: #007bff;
            color: white;
            border: none;
            border-radius: 5px;
```

```
            cursor: pointer;
        }

        button:hover {
            background-color: #0056b3;
        }

        #message {
            text-align: center;
            margin-top: 15px;
            font-weight: bold;
        }
    </style>
</head>

<body>

    <div class="login-box">
        <h2>Login</h2>

        <form id="loginForm">

            <label for="username">Username</label>
            <input type="text" id="username" placeholder="Enter username">

            <label for="password">Password</label>
            <input type="password" id="password" placeholder="Enter password">

            <button type="submit">Login</button>
          <p id="message"></p>

        </form>
    </div>

    <script>
        document.getElementById("loginForm").addEventListener("submit", function(event) {

            event.preventDefault();

            let username = document.getElementById("username").value;
            let password = document.getElementById("password").value;
            let message = document.getElementById("message");

            // Check blank fields
            if (username === "" || password === "") {
                message.textContent = "Please enter username and password.";
                message.style.color = "red";
                return;
            }
```

```
            // Valid login credentials
            if (username === "admin" && password === "12345") {
                message.textContent = "Login successful!";
                message.style.color = "green";

                // Redirect after successful login
                setTimeout(function() {
                    window.location.href = "home.html";
                }, 1000);

            } else {
                message.textContent = "Invalid username or password.";
                message.style.color = "red";
            }
        });
    </script>
</body>
</html>
```

Output

Step 2:Create Excel Worksheet
Create Manual Testing .xlsx
Add 5 Sheets:
1.Test plan
2.Test Scenarios
3.Test Cases
4.Test Execution Report
5.Defect Report

1.Test plan

2.Test Scenarios

3.Test Cases

4.Test Execution Report

5.Defect Report

PRACTICAL NO-4

AIM: Install Selenium Server(Selenium RC) and demonstrate it using a script in Java.

1) To Download "JDK":

- Visit https://www.oracle.com/technetwork/java/javase/downloads/jdk12-downloads-5295953.html
- Download this file "jdk-12.0.2_windows-x64_bin.exe" and install it.

2) To Download "Eclipse IDE":

- Visit https://www.eclipse.org/downloads/download.php?file=/oomph/epp/2019-06/R/eclipse-inst-win64.exe
- Click "Download".
- Installation:
- Open application.(click OK if errors occur like "could not find java.dll" & "could not find Java Runtime Environment SE")
- Select "Eclipse IDE for Java Developers".
- It will automatically locate the JDK. Choose path, and click "Install".
- After installation, click "Launch" or open Eclipse from START menu in Windows.

3) To Download "Selenium Server Driver and Client Driver(JAR files)":

a) For "Selenium Server Driver":

- Visit https://www.seleniumhq.org/download/
- Under section "Selenium Standalone Server", click download version "3.141.59"
- You'll get the executable jar file(selenium-server-standalone-3.141.59)

b) For "Selenium Client Driver":

- Visit https://www.seleniumhq.org/download/
- Under section "Selenium Client & WebDriver Language Bindings", download the "3.141.59" version of Java.
- Extract the file and you'll see two jar files. From them, we'll be using this
- executable jar file(client-combined-3.141.59)

4) To Download "Gecko Driver":

- Visit https://github.com/mozilla/geckodriver/releases

STEPS:

1) Open Eclipse. Select your workspace directory. Click Launch:

2) Create a Project(File > New > Java Project):

3) Name the project as "gcd" > click Finish > click Don't Create module:

4) Close the "Welcome" tab.

5) Create a Package(right-click on Project Name > New > Package > Name it > Finish):

6) Create a Class(right-click on Project Name > New > Class > Name it > Finish):

7) Adding "Selenium Server Driver and Client Driver(JAR files)" in Eclipse IDE:

• right-click on Project Name > Build Path > Configure Build Path...

• now go under: Java Build Path > Libraries > Classpath > click Add External JARs…

Code

```
import org.openqa.selenium.By;
import org.openqa.selenium.WebDriver;
import org.openqa.selenium.firefox.FirefoxDriver;
import org.openqa.selenium.remote.DesiredCapabilities;

public class Demo {

        public static void main(String[] args) {
```

```
        //Creating a driver object referencing WebDriver interfaceWebDriver
        driver;

        //Setting webdriver.gecko.driver property
        System.setProperty("webdriver.gecko.driver",
"C:\\Users\\106\\Desktop\\STQA\\geckodriver.exe");

        //Instantiating driver object and launching browserdriver = new
        FirefoxDriver();

        //Using get() method to open a webpage
        driver.get("https://google.com");

        //Closing the browser
        driver.quit();
    }}
```

OUTPUT

PRACTICAL NO-5

AIM: Write a program using Selenium WebDriver to automate the login process on a specific web page. Verify Successful login with appropriate assertions.

PRE-REQUISITES:

1) Check that you have JDK.

2) Check that you have Eclipse IDE.

3) Check that you have Selenium Server Driver and Client Driver(JAR files).

4) Check that you have Gecko Driver.

5) Check that you have a stable Internet connection.

STEPS

1) Open Eclipse. Select your workspace directory. Click Launch:

2) Create a Project(File > New > Java Project):

3) Name the project as "gcd" > click Finish > click Don't Create module:

4) Close the "Welcome" tab.

5) Create a Package(right-click on Project Name > New > Package > Name it > Finish):

6) Create a Class(right-click on Project Name > New > Class > Name it > Finish):

7) Adding "Selenium Server Driver and Client Driver(JAR files)" in Eclipse IDE:

• right-click on Project Name > Build Path > Configure Build Path…

• **now go under: Java Build Path > Libraries > Classpath > click Add External JARs…**

Code:

```
package p7;
import org.openqa.selenium.By;
import org.openqa.selenium.Keys;
import org.openqa.selenium.WebDriver;
import org.openqa.selenium.WebElement;
import org.openqa.selenium.firefox.FirefoxDriver;
import org.openqa.selenium.firefox.*;
import org.openqa.selenium.firefox.FirefoxOptions;
import org.openqa.selenium.firefox.FirefoxProfile;
import org.openqa.selenium.firefox.internal.ProfilesIni;
public class FB_login {
static String driverPath = "C:\\Users\\Usman\\503\\geckodriver.exe";
public static void main(String[] args) {
System.setProperty("webdriver.gecko.driver", driverPath);
//DesiredCapabilities capabilities = DesiredCapabilities.firefox();
//capabilities.setCapability("marionette",true
); ProfilesIni allProfiles = new ProfilesIni();
FirefoxProfile fp = new FirefoxProfile();
fp.setPreference(FirefoxProfile.PORT_PREFERENCE,"7055");
FirefoxOptions options = new FirefoxOptions();
options.setProfile(fp);
//objects and variables instantiation
WebDriver driver = new FirefoxDriver(options);
String appUrl = "https://www.facebook.com/";
```

```
//launch the firefox browser and open the application url
driver.get(appUrl);
//maximize the browser window
driver.manage().window().maximize();
//declare and initialize the variable to store the expected title of the
webpage.
String expectedTitle = "Facebook – log in or sign up";
//fetch the title of the web page and save it into a string variable
String actualTitle = driver.getTitle();
//compare the expected title of the page with the actual title of the page
and print the result
if (expectedTitle.equals(actualTitle)) {
System.out.println("Verification Successful - The correct title is
displayed on the web page.");
}
else {
System.out.println("Verification Failed - An incorrect title is
displayed on the web page.");
}
//enter a valid username in the email textbox
WebElement username = driver.findElement(By.id("email"));
username.clear();
username.sendKeys("abc@gmail.com");
//enter a valid password in the password textbox
WebElement password = driver.findElement(By.id("pass"));
password.clear();
password.sendKeys("Abc@123");
```

```
password.sendKeys(Keys.ENTER);
//click on the Sign in button
WebElement LogInButton =
driver.findElement(By.id("u_0_2")); LogInButton.click();
//close the web browser
driver.close();
System.out.println("Test script executed successfully.");
//terminate the program
System.exit(0);
}
}
```

OUTPUT:

PRACTICAL NO-6

AIM: Write and test a program to update 10 student records into Excel file(table).

STEPS

1) Open Eclipse. Select your workspace directory. Click Launch:

2) Create a Project(File > New > Java Project):

3) Name the project as "gcd" > click Finish > click Don't Create module:

4) Close the "Welcome" tab.

5) Create a Package(right-click on Project Name > New > Package > Name it > Finish):

6) Create a Class(right-click on Project Name > New > Class > Name it > Finish):

7) Adding "Selenium Server Driver and Client Driver(JAR files)" in Eclipse IDE:

• right-click on Project Name > Build Path > Configure Build Path…

• now go under: Java Build Path > Libraries > Classpath > click Add External JARs…

Code:

```
package excelwrite;
import jxl.*; //used for WorkbookSettings,Workbook
import jxl.write.*; //used for WriteException,WritableWorkbook,WritableSheet,Label
import jxl.write.Number; //used for Number
import java.io.*; //used for IOException,File
import java.util.Locale; //used for Locale
public class Excelwriter {
public static void main(String[] args) throws IOException,WriteException {
// TODO Auto-generated method stub
int r=0,c=0;
String header[]={"Student
Name","Subject1","Subject2","Subject3","Total"
};
Strings name[]={"Carls","James","Paul","Philip","Smith","Thomson","Rhodey",
"Stark","Gary","AnneMarie"};
int marks[]={50,45,60,55,70,45,67,78,89,90,30};
File file = new File("student.xls");
WorkbookSettings wbSettings = new WorkbookSettings();
wbSettings.setLocale(new Locale("en", "EN"));
WritableWorkbook workbook =
Workbook.createWorkbook(file,wbSettings);
workbook.createSheet("Report", 0);
WritableSheet excelSheet = workbook.getSheet(0);
//creating header row
```

```
for(r=0;r<1;r++) {
for(c=0;c<header.length;c++)
Label l=new Label(c,r,header[c]);
excelSheet.addCell(l);
}
}
//filling name in column1
for(r=1;r<=sname.length;r++) {
for(c=0;c<1;c++) {
Label l=new Label(c,r,sname[r-1]);
excelSheet.addCell(l);
}
}
//filling name in column2,3,4
for(r=1;r<=sname.length;r++) {
for(c=1;c<4;c++) {
Number num = new Number(c, r, marks[r-1]);
excelSheet.addCell(num);
}
}
//filling name in total
for(r=1;r<=sname.length;r++) {
for(c=4;c<5;c++) {
int total=marks[r-1]+marks[r-1]+marks[r-1];
Number num = new Number(c, r, total);
excelSheet.addCell(num);
}
```

```
}
workbook.write();

workbook.close();

System.out.println("Excel File Created!!!!!");

}

}
```

OUTPUT:

PRACTICAL NO-7

AIM: Write and test a program to select the number of students from Excel file(table) who have scored 60 or more in any one subject(or all subjects).

PRE-REQUISITES:

1) Check that you have Eclipse IDE.

2) Check that you have JXL.JAR.

3) Check that you've the Excel file("student.xls") that we'll be working on for reading data.

STEPS

1) Open Eclipse. Select your workspace directory. Click Launch:

2) Create a Project(File > New > Java Project):

3) Name the project as "gcd" > click Finish > click Don't Create module:

4) Close the "Welcome" tab.

5) Create a Package(right-click on Project Name > New > Package > Name it > Finish):

6) Create a Class(right-click on Project Name > New > Class > Name it > Finish):

7) Adding "Selenium Server Driver and Client Driver(JAR files)" in Eclipse IDE:

• right-click on Project Name > Build Path > Configure Build Path…

• **now go under: Java Build Path > Libraries > Classpath > click Add External JARs…**

Code:

```
package excelread;
import java.io.File;
import java.io.IOException;
import jxl.Cell;
import jxl.CellType;
import jxl.Sheet;
import jxl.Workbook;
import jxl.read.biff.BiffException;
public class Excelreader {
private String inputFile;
public void setInputFile(String inputFile) {this.inputFile = inputFile;}
public void read() throws IOException {
File inputWorkbook = new File(inputFile);
Workbook w;
boolean flag=false;
int count=0;
try {
w = Workbook.getWorkbook(inputWorkbook);
// Get the first sheet
Sheet sheet = w.getSheet(0);
// Loop over first 10 column and lines
for (int j = 0; j < sheet.getRows(); j++)
{
for (int i = 0; i < sheet.getColumns()-1; i++) {
```

```
Cell cell = sheet.getCell(i, j);

if (cell.getType() == CellType.NUMBER) {

if(Integer.parseInt(cell.getContents())>=60){

flag = true;

if(flag == true){

count++;

flag=false;

}

break;

}

  }

     }

}

System.out.println("Total number of students who scored morethan 60 in one or more
subjects: " +count);

}

catch (BiffException e) {e.printStackTrace();}

}

public static void main(String[] args) throws IOException {

Excelreader test = new Excelreader();

test.setInputFile("C:\\Users\\Usman\\eclipse-

workspace\\p5\\student.xls"); test.read();

}

}
```

OUTPUT:

PRACTICAL NO-8

AIM: Write and test a program to provide total number of objects present/available on webpage.

PRE-REQUISITES:

1) Check that you have JDK.

2) Check that you have Eclipse IDE.

3) Check that you have Selenium Server Driver and Client Driver(JAR files).

4) Check that you have Gecko Driver.

5) Check that you have a stable Internet connection.

STEPS

1) Open Eclipse. Select your workspace directory. Click Launch:

2) Create a Project(File > New > Java Project):

3) Name the project as "gcd" > click Finish > click Don't Create module:

4) Close the "Welcome" tab.

5) Create a Package(right-click on Project Name > New > Package > Name it > Finish):

6) Create a Class(right-click on Project Name > New > Class > Name it > Finish):

7) Adding "Selenium Server Driver and Client Driver(JAR files)" in Eclipse IDE:

• right-click on Project Name > Build Path > Configure Build Path…

• now go under: Java Build Path > Libraries > Classpath > click Add External JARs… Code:

```
package p8;
import org.openqa.selenium.By;
import org.openqa.selenium.WebDriver;
import org.openqa.selenium.WebElement;
import org.openqa.selenium.firefox.FirefoxDriver;
import org.openqa.selenium.firefox.*;
import org.openqa.selenium.firefox.FirefoxOptions;
import org.openqa.selenium.firefox.FirefoxProfile;
import org.openqa.selenium.firefox.internal.ProfilesIni;
public class FindAllLinks {
static String driverPath = "C:\\Users\\Usman\\503\\geckodriver.exe";
public static void main(String[] args) {
System.setProperty("webdriver.gecko.driver",driverPath);
//NOTE THAT: following commented lines are required for old machines
//DesiredCapabilities capabilities = DesiredCapabilities.firefox();
//capabilities.setCapability("marionette",true);
//ProfilesIni allProfiles = new ProfilesIni();
//FirefoxProfile fp = new FirefoxProfile();
//fp.setPreference(FirefoxProfile.PORT_PREFERENCE,"7055");
//FirefoxOptions options = new FirefoxOptions();
//options.setProfile(fp);
WebDriver driver = new FirefoxDriver();
String appUrl ="https://www.google.co.in/";
driver.get(appUrl);
java.util.List<WebElement> links =driver.findElements(By.tagName("a"));
```

```
for (int i = 1; i<links.size(); i=i+1)
{
System.out.println(links.get(i).getText());
}
System.out.println("Total No. of Links: "+links.size());
//driver.quit();
}
```

OUTPUT

PRACTICAL NO-9

AIM: Write and test a program to get the number of items in a list/combo box. PRE-REQUISITES:

1) Check that you have JDK.

2) Check that you have Eclipse IDE.

3) Check that you have Selenium Server Driver and Client Driver(JAR files).

4) Check that you have Gecko Driver.

5) Check that you have a stable Internet connection.

STEPS

1) Open Eclipse. Select your workspace directory. Click Launch:

2) Create a Project(File > New > Java Project):

3) Name the project as "gcd" > click Finish > click Don't Create module:

4) Close the “Welcome” tab.

5) Create a Package(right-click on Project Name > New > Package > Name it > Finish):

6) Create a Class(right-click on Project Name > New > Class > Name it > Finish):

7) Adding “Selenium Server Driver and Client Driver(JAR files)” in Eclipse IDE:

• right-click on Project Name > Build Path > Configure Build Path…

• now go under: Java Build Path > Libraries > Classpath > click Add External JARs…

8) Create HTML file in a notepad > Save it > Open it in browser > Copy the URL:

(combobox.html)

```
<select id="continents">
<option value="Asia"> Asia </option>
<option value="Europe"> Europe </option>
```

<option value="Africa"> Africa </option>

</select>

Code:

```
import java.util.List;
import org.openqa.selenium.By;
import org.openqa.selenium.WebDriver;
import org.openqa.selenium.WebElement;
import org.openqa.selenium.firefox.FirefoxDriver;
import org.openqa.selenium.firefox.*;
import org.openqa.selenium.firefox.FirefoxOptions;
import org.openqa.selenium.firefox.FirefoxProfile;
import org.openqa.selenium.firefox.internal.ProfilesIni;
import org.openqa.selenium.support.ui.Select;
public class ComboBox {
static String driverPath = "C:\\Users\\Usman\\503\\geckodriver.exe";
public static void main(String[] args) {
System.setProperty("webdriver.gecko.driver",driverPath);
//NOTE THAT: following commented lines are required for old machines
//DesiredCapabilities capabilities = DesiredCapabilities.firefox();
//capabilities.setCapability("marionette",true);
//ProfilesIni allProfiles = new ProfilesIni();
//FirefoxProfile fp = new FirefoxProfile();
//fp.setPreference(FirefoxProfile.PORT_PREFERENCE,"7055");
//FirefoxOptions options = new FirefoxOptions();
//options.setProfile(fp);
WebDriver driver = new FirefoxDriver();
String appUrl ="https://www.toolsqa.com/automation-practice-form/";
```

```
//DYNAMIC URL(WEBSITE)

//String appUrl = "file:///D:/Usman/College/503%20pracs/combobox.html"; //STATIC
URL(LOCAL FILE)

driver.get(appUrl);

Select oSelect =
new

Select(driver.findElement(By.id("continents")))

//Select oSelect = new Select(driver.findElement(By.tagName("select"))); //this works too

List<WebElement> oSize = oSelect.getOptions();

int iListSize = oSize.size();

for(int i =0; i < iListSize ; i++)

{

// Storing the value of the option

String sValue = oSelect.getOptions().get(i).getText();

// Printing the stored value

System.out.println(sValue);

}

System.out.println("Total No. Items in Dropdown: "+iListSize);

//driver.quit();

}}
```

OUTPUT

Model-2

PRACTICAL NO-1

AIM: Write and test a program to count the number of check boxes on the page checked and unchecked count.

PRE-REQUISITES:

1) Check that you have JDK.

2) Check that you have Eclipse IDE.

3) Check that you have Selenium Server Driver and Client Driver(JAR files).

4) Check that you have Gecko Driver.

5) Check that you have a stable Internet connection.

1) Open Eclipse. Select your workspace directory. Click Launch:

2) Create a Project(File > New > Java Project):

3) Name the project as "gcd" > click Finish > click Don't Create module:

4) Close the "Welcome" tab.

5) Create a Package(right-click on Project Name > New > Package > Name it > Finish):

6) Create a Class(right-click on Project Name > New > Class > Name it > Finish):

7) Adding "Selenium Server Driver and Client Driver(JAR files)" in Eclipse IDE:

• right-click on Project Name > Build Path > Configure Build Path…

• now go under: Java Build Path > Libraries > Classpath > click Add External JARs…

8) Create HTML file in a notepad > Save it > Open it in browser > Copy the URL:

(multicheckbox.html)

```
<input type="checkbox" value="A">A<br>
<input type="checkbox" value="B" CHECKED>B<br>
<input type="checkbox" value="C">C<br>
<input type="checkbox" value="D" CHECKED>D<br>
<input type="checkbox" value="E">E<br>
```

Code:

```
import java.util.List;
import org.openqa.selenium.By;
import org.openqa.selenium.WebDriver;
import org.openqa.selenium.WebElement;
import org.openqa.selenium.firefox.FirefoxDriver;
import org.openqa.selenium.firefox.*;
import org.openqa.selenium.firefox.FirefoxOptions;
import org.openqa.selenium.firefox.FirefoxProfile;
import org.openqa.selenium.firefox.internal.ProfilesIni;
public class MultiCheckBox {
static String driverPath = "C:\\Users\\Usman\\503\\geckodriver.exe";
public static void main(String[] args) {
System.setProperty("webdriver.gecko.driver",driverPath);
//DesiredCapabilities capabilities = DesiredCapabilities.firefox();
//capabilities.setCapability("marionette",true);
//ProfilesIni allProfiles = new ProfilesIni();
//FirefoxProfile fp = new FirefoxProfile();
//fp.setPreference(FirefoxProfile.PORT_PREFERENCE,"7055");
```

```
//FirefoxOptions options = new FirefoxOptions();

//options.setProfile(fp);

WebDriver driver = new FirefoxDriver

String appUrl = "https://www.toolsqa.com/automation-practice-form/";

//DYNAMIC URL(WEBSITE)

//String appUrl ="file:///D:/Usman/College/503%20pracs/multicheckbox.html"; //STATIC
URL(LOCALFILE)

driver.get(appUrl);

List<WebElement> checkBoxes =

driver.findElements(By.xpath("//input[@type='checkbox']")

); int checkedCount=0, uncheckedCount=0;

for(int i=0; i<checkBoxes.size(); i++){

System.out.println(i+" checkbox is selected

"+checkBoxes.get(i).isSelected());

if(checkBoxes.get(i).isSelected()) {checkedCount++;}

else {uncheckedCount++;}

}

//driver.quit();

System.out.println("No. of selected checkbox: "+checkedCount);

System.out.println("No. of unselected checkbox: "+uncheckedCount);

}

}
```

OUTPUT

PRACTICAL NO - 2

AIM : To write and test a Selenium WebDriver program to extract data from a dynamic web table, validate the sorting functionality, and verify the correctness of filtered results.

Tool Required

- Java
- Eclipse IDE
- Selenium WebDriver
- Chrome Browser
- ChromeDriver

Website Used

https://demo.opencart.com/admin/ *(or any dynamic table website)*

Alternative Demo Site (Recommended):
https://the-internet.herokuapp.com/tables

```
import java.util.ArrayList;
import java.util.Collections;
import java.util.List;

import org.openqa.selenium.By;
import org.openqa.selenium.WebDriver;
import org.openqa.selenium.WebElement;
import org.openqa.selenium.chrome.ChromeDriver;

public class DynamicWebTable {

    public static void main(String[] args) throws InterruptedException {

        // Set ChromeDriver Path
       // System.setProperty("webdriver.chrome.driv   "C:\\Drivers\\chromedriver.exe");

        WebDriver driver = new ChromeDriver();

        // Open Website
        driver.get("https://the-internet.herokuapp.com/tables");
        driver.manage().window().maximize();
```

```
        Thread.sleep(2000);

        // Extract Data from First Table
        List<WebElement> rows = driver.findElements(By.xpath("//table[@id='table1']/tbody/tr"));

        System.out.println("Table Data:");

        for (WebElement row : rows) {
            List<WebElement> cols = row.findElements(By.tagName("td"));

            for (WebElement col : cols) {
                System.out.print(col.getText() + "\t");
            }
            System.out.println();
        }

        // Click Last Name column for sorting
        driver.findElement(By.xpath("//table[@id='table1']//span[text()='Last Name']")).click();

        Thread.sleep(2000);

        // Read sorted values
        List<WebElement> sortedRows =
driver.findElements(By.xpath("//table[@id='table1']/tbody/tr/td[1]"));

        List<String> actualList = new ArrayList<>();

        for (WebElement e : sortedRows) {
            actualList.add(e.getText());
        }

        List<String> expectedList = new ArrayList<>(actualList);
        Collections.sort(expectedList);

        System.out.println("\nSorting Validation:");

        if (actualList.equals(expectedList)) {
            System.out.println("Table is sorted correctly.");
        } else {
            System.out.println("Sorting Failed.");
        }

        driver.quit();
    }
```

```
}
```

PRACTICAL NO-3

AIM:To develop a Selenium WebDriver script to automate file upload and download functionality, validate successful upload, and verify the existence of the downloaded file.

Tool: Selenium WebDriver + Robot Class
Language: Java
IDE: Eclipse

Requirements

- Java JDK
- Eclipse IDE
- Selenium WebDriver
- Chrome Browser
- ChromeDriver
- Java Robot class

Program

```
package jayraj;

import java.io.File;

import org.openqa.selenium.By;

import org.openqa.selenium.WebDriver;

import org.openqa.selenium.chrome.ChromeDriver;

public class prat1 {

    public static void main(String[] args) throws Exception {

        // ==========================

        // OPEN CHROME

        // ==========================

        WebDriver driver = new ChromeDriver();

        driver.manage().window().maximize();

        // ==========================

        // FILE UPLOAD

        // ==========================

        driver.get("https://the-internet.herokuapp.com/upload");

        Thread.sleep(2000);

        // Directly select file using Selenium

        driver.findElement(By.id("file-upload"))
```

```
        .sendKeys("C:\\model\\tycs.txt");
Thread.sleep(1000);
// Click Upload button
driver.findElement(By.id("file-submit")).click();
Thread.sleep(3000);
// Validate Upload
String msg = driver.findElement(By.tagName("h3")).getText();
if (msg.equals("File Uploaded!")) {
    System.out.println("File Uploaded Successfully");
} else {
    System.out.println("Upload Failed");
}
// ==========================
// FILE DOWNLOAD
// ==========================
driver.get("https://the-internet.herokuapp.com/download");
Thread.sleep(2000);
// Click sample file
driver.findElement(By.linkText("some-file.txt")).click();
Thread.sleep(5000);
// ==========================
// VERIFY DOWNLOAD
// ==========================
File downloadedFile= new File("C:\\Users\\YourUserName\\Downloads\\some-file.txt");
if (downloadedFile.exists()) {
    System.out.println("File Downloaded Successfully");
} else {
    System.out.println("Download Failed");
}
// ==========================
// CLOSE BROWSER
// ==========================
```

```
        driver.quit();
    }
}
```

PRACTICAL No-4

AIM:To implement screenshot capture on test failure and integrate logging using Log4j with Selenium WebDriver.

Tool: Selenium WebDriver + Log4j

Program.java

```
package Pract;

import java.io.File;

import org.apache.logging.log4j.LogManager;
import org.apache.logging.log4j.Logger;

import org.openqa.selenium.By;
import org.openqa.selenium.OutputType;
import org.openqa.selenium.TakesScreenshot;
import org.openqa.selenium.WebDriver;
import org.openqa.selenium.chrome.ChromeDriver;

import org.testng.Assert;
import org.testng.annotations.AfterMethod;
import org.testng.annotations.BeforeMethod;
import org.testng.annotations.Test;

public class Program {

    WebDriver driver;

    // Log4j
    Logger log = LogManager.getLogger(Program.class);

    @BeforeMethod
    public void startBrowser() {

        driver = new ChromeDriver();
        driver.manage().window().maximize();

        log.info("Browser started");

        driver.get("https://www.google.com");

        log.info("Website opened");
    }

    @Test
    public void testFailure() {

        try {

            log.info("Test started");
```

```
            // Wrong ID - creates failure
            driver.findElement(By.id("wrong_id")).click();

            log.info("Test passed");

        } catch (Exception e) {

            log.error("Test failed");
            log.error("Error: " + e.getMessage());

            // Screenshot
            TakesScreenshot ts =
                    (TakesScreenshot) driver;

            File source =
                    ts.getScreenshotAs(OutputType.FILE);

            File destination =
                    new File("failure.png");

            source.renameTo(destination);

            log.info("Screenshot captured");

            Assert.fail("Test failed");
        }
    }

    @AfterMethod
    public void closeBrowser() {

        driver.quit();

        log.info("Browser closed");
    }
}
```

Logj2.xml

```
<?xml version="1.0" encoding="UTF-8"?>

<Configuration>

    <Appenders>

        <Console name="Console">
            <PatternLayout pattern="%d %-5level %msg%n"/>
        </Console>
        <File name="File" fileName="execution.log">
            <PatternLayout pattern="%d %-5level %msg%n"/>
```

```
        </File>
    </Appenders>

    <Loggers>

        <Root level="info">
            <AppenderRef ref="Console"/>
            <AppenderRef ref="File"/>
        </Root>

    </Loggers>

</Configuration>
```

PRACTICAL NO-5

AIM:To design and implement an automation testing framework using Page Object Model (POM) with Selenium WebDriver and TestNG, by creating reusable page classes and executing modular test cases.

2. Project Structure

Create the project like this:

POMProject

```
│
├── pages
│   └── LoginPage.java
│
├── tests
│   └── LoginTest.java
│
└── testng.xml
```

In Eclipse:

1. Create Java/Maven project.
2. Add **Selenium WebDriver** dependency.
3. Add **TestNG** dependency/plugin.
4. Create pages package.
5. Create LoginPage.java
6. Create tests package.
7. Create LoginTest.java.
8. Create testng.xml.
9. Right-click testng.xml.
10. Select **Run As → TestNG Suite**

3. LoginPage.java

```
package pages;

import org.openqa.selenium.By;
import org.openqa.selenium.WebDriver;

public class LoginPage {

    WebDriver driver;

    By username = By.id("username");
    By password = By.id("password");
    By login = By.id("login-button");

    public LoginPage(WebDriver driver) {
        this.driver = driver;
    }

    public void enterUsername(String user) {
        driver.findElement(username).sendKeys(user);
    }

    public void enterPassword(String pass) {
        driver.findElement(password).sendKeys(pass);
    }

    public void clickLogin() {
        driver.findElement(login).click();
    }
}
```

3. LoginTest.java

```
package tests;

import org.openqa.selenium.WebDriver;
import org.openqa.selenium.chrome.ChromeDriver;
import org.testng.annotations.AfterMethod;
import org.testng.annotations.BeforeMethod;
import org.testng.annotations.Test;

import pages.LoginPage;
```

```
public class LoginTest {

    WebDriver driver;
    LoginPage loginPage;

    @BeforeMethod
    public void setup() {

        driver = new ChromeDriver();
        driver.manage().window().maximize();

        driver.get("https://www.saucedemo.com");

        loginPage = new LoginPage(driver);
    }

    @Test
    public void loginTest() {

        loginPage.enterUsername("standard_user ");
        loginPage.enterPassword("secret_sauce ");
        loginPage.clickLogin();

        System.out.println("Login Test Executed Successfully");
    }

    @AfterMethod
    public void close() {

        driver.quit();
    }
}
```

4. testng.xml

```
<?xml version="1.0" encoding="UTF-8"?>

<!DOCTYPE suite SYSTEM "https://testng.org/testng-1.0.dtd">

<suite name="POM Suite">

    <test name="Login Test">
        <classes>
            <class name="Tests.LoginTests"/>
```

```
        </classes>
    </test>

</suite>
```

PRACTICAL NO-6

AIM:To create and execute automated test cases using TestNG annotations, group test cases, define priorities, and generate HTML test reports using Selenium WebDriver.

1. Required Software

- Eclipse IDE
- Java JDK
- Selenium WebDriver
- TestNG
- Google Chrome
- ChromeDriver / Selenium Manager

2. Project Structure

```
TestNGDemo
 │
├── src/test/java
│    └── TestNGDemo.java
│
└── testng.xml
```

TestNGDemo.java

```
package practical02;
import org.openqa.selenium.By;
import org.openqa.selenium.WebDriver;
import org.openqa.selenium.chrome.ChromeDriver;
import org.testng.Assert;
import org.testng.annotations.AfterClass;
import org.testng.annotations.BeforeClass;
import org.testng.annotations.Test;

public class Practical2 {

    WebDriver driver;

    @BeforeClass
    public void setup() {
        driver = new ChromeDriver();
        driver.manage().window().maximize();
    }

    @Test(priority = 1, groups = {"smoke"})
    public void openWebsite() {
        driver.get("https://www.google.com");
        String title = driver.getTitle();
```

```
        System.out.println("Page Title: " + title);
        Assert.assertTrue(title.contains("Google"));
    }

    @Test(priority = 2, groups = {"smoke", "search"})
    public void searchTest() {
        driver.get("https://www.google.com");
        driver.findElement(By.name("q"))
                .sendKeys("Selenium WebDriver");
        driver.findElement(By.name("q"))
                .submit();
        System.out.println("Search test executed");
        Assert.assertTrue(driver.getTitle().contains("Selenium"));
    }

    @Test(priority = 3, groups = {"regression"})
    public void titleTest() {
        driver.get("https://www.google.com");
        String title = driver.getTitle();
        System.out.println("Title Test: " + title);
        Assert.assertEquals(title, "Google");
    }

    @AfterClass
    public void closeBrowser() {
        //driver.quit();
    }
}
```

Testng.xml

```
<?xml version="1.0" encoding="UTF-8"?>
<!DOCTYPE suite SYSTEM
"https://testng.org/testng-1.0.dtd">

<suite name="Selenium Test Suite">
    <test name="Smoke Tests">
        <groups>
            <run>
                <include name="smoke"/>
            </run>
        </groups>

        <classes>
            <class name="practical02.Practical2"/>
        </classes>
    </test>
</suite>
```

Add selenium-server-4.45.0 Jar

Step 1 — Install TestNG

In Eclipse:

Help → Eclipse Marketplace

Search for:

TestNG for Eclipse

Click **Install**, then restart Eclipse if requested.

Step 2 — Add TestNG to your project

After restarting:

9) Right-click your project.
10) Select **TestNG → Convert to TestNG** if available.

Or:

Right-click Project → Build Path → Add Libraries → TestNG → Next → Finish

Output

PRACTICAL NO-7

AIM:To configure Jenkins to execute Selenium automation scripts automatically and generate build reports to analyze test execution history.

Tool: Jenkins + Selenium WebDriver

Requirements

- Java JDK
- Eclipse IDE
- Selenium WebDriver
- TestNG
- Jenkins
- Google Chrome
- ChromeDriver / Selenium Manager

LoginTest.java

```
package tests;

import org.openqa.selenium.By;
import org.openqa.selenium.WebDriver;
import org.openqa.selenium.chrome.ChromeDriver;
import org.testng.Assert;
import org.testng.annotations.Test;

public class LoginTest {

    @Test
    public void loginTest() throws InterruptedException {

        WebDriver driver = new ChromeDriver();

        driver.get("file:///C:/Users/model9/Desktop/login.htm");

        driver.findElement(By.id("username")).sendKeys("admin");
        driver.findElement(By.id("password")).sendKeys("1234");

        driver.findElement(By.id("login")).click();

        Thread.sleep(2000);

        String title = driver.getTitle();

        System.out.println("Login Test Executed Successfully");
```

```
        System.out.println("Page Title: " + title);
        Assert.assertTrue(true);

        driver.quit();
    }
}
```

testng.xml

```
<?xml version="1.0" encoding="UTF-8"?>

<!DOCTYPE suite SYSTEM "https://testng.org/testng-1.0.dtd">

<suite name="Selenium Test Suite">

    <test name="Login Test">

        <classes>
            <class name="tests.LoginTest"/>
        </classes>

    </test>

</suite>
```

Output

PRACTICAL No- 8

AIM: To execute Selenium automation scripts across multiple browsers using Selenium Grid and compare execution time and compatibility issues.

Requirements

- Java
- Eclipse/IntelliJ
- Selenium WebDriver
- Selenium Grid 4
- Chrome
- Firefox
- Microsoft Edge

Code

```
import java.net.URL;
import org.openqa.selenium.By;
import org.openqa.selenium.WebDriver;
import org.openqa.selenium.chrome.ChromeOptions;
import org.openqa.selenium.edge.EdgeOptions;
import org.openqa.selenium.firefox.FirefoxOptions;
import org.openqa.selenium.remote.RemoteWebDriver;

public class CrossBrowserTest {

    public static void main(String[] args) {

        String gridURL = "http://localhost:4444";

        // Chrome Test
        runTest("chrome", gridURL);

        // Firefox Test
        runTest("firefox", gridURL);

        // Microsoft Edge Test
        runTest("MicrosoftEdge", gridURL);
    }

    public static void runTest(String browser, String gridURL) {

        WebDriver driver = null;

        try {

            // Start execution timer
            long startTime = System.currentTimeMillis();
```

```
// Launch Chrome
if (browser.equalsIgnoreCase("chrome")) {

    ChromeOptions options = new ChromeOptions();

    driver = new RemoteWebDriver(
            new URL(gridURL), options
    );
}

// Launch Firefox
else if (browser.equalsIgnoreCase("firefox")) {

    FirefoxOptions options = new FirefoxOptions();

    driver = new RemoteWebDriver(
            new URL(gridURL), options
    );
}

// Launch Microsoft Edge
else if (browser.equalsIgnoreCase("MicrosoftEdge")) {

    EdgeOptions options = new EdgeOptions();

    driver = new RemoteWebDriver(
            new URL(gridURL), options
    );
}

// Open Google
driver.get("https://www.google.com");

// Display page title
System.out.println(
        browser + " - Page Title: "
        + driver.getTitle()
);

// Search for Selenium Grid
driver.findElement(By.name("q"))
        .sendKeys("Selenium Grid");

// Calculate execution time
long endTime = System.currentTimeMillis();

System.out.println(
        browser + " - Execution Time: "
        + (endTime - startTime) + " ms"
);
```

```
            // Close browser
            driver.quit();

        } catch (Exception e) {

            System.out.println(
                    browser + " - Test Failed"
            );

            e.printStackTrace();

            // Close browser if an error occurs
            if (driver != null) {
                driver.quit();
            }
        }
    }
}
```

Output